1. Explain Abstraction in Java

Abstraction hides implementation details and shows only essential features using abstract classes or interfaces.

Example: `abstract class Vehicle { abstract void start(); }` – you know vehicle starts, but how depends on the subclass like `Car` or `Bike`.

2. Differentiate between Abstract Class and Interface

Abstract Class	**Interface**
Can have instance variables & constructors	Only static & final variables (by default)
Methods can be abstract or concrete	All methods are abstract (before Java 8); default/static methods allowed later
A class extends only one abstract class	A class implements multiple interfaces
Use `abstract` keyword	Use `interface` keyword
Example: `abstract class Animal { abstract void sound(); void sleep() {}}`	Example: `interface Playable { void play(); }`

3. Upcasting and Dynamic Method Dispatch

- **Upcasting:** Assigning subclass object to superclass reference – done implicitly.
  `Animal a = new Dog();`
- **Dynamic Method Dispatch:** At runtime, Java decides which overridden method to call based on the actual object type, not reference type.

java

```
class Animal { void sound() { System.out.println("Animal"); } }
class Dog extends Animal { void sound() { System.out.println("Bark"); } }

public class Main {
    public static void main(String[] args) {
        Animal a = new Dog(); // Upcasting
```

```
        a.sound(); // Output: Bark (Dynamic dispatch)
    }
}
```
